EngineSquare

보안 관제 및 SOAR 자동 대응 시스템

개인 포트폴리오 정리

개인 저장소: github.com/rykim1217/network-team03

팀 원본: github.com/altdmfk/network-team03

이름: 김라연 · 기간: 2026.10.20~2026.11.11 (약 3주)

1. 프로젝트 개요

웹/API 보안 이벤트를 수집하고 Graylog·Wazuh로 탐지한 뒤 n8n과 API·Discord·pfSense로 자동 대응하는 통합 관제 프로젝트입니다.

구성 흐름

입력	분석	대응
API·공격 트래픽·IoT 이벤트	Middleware WAL → Graylog/Wazuh → 상관분석	n8n → API 차단·키 교체·Discord·pfSense·대시보드

시나리오

시나리오	핵심 기능	개인 검증
1. 브루트포스	로그 버퍼링·401 집계·IP 차단	Alert·n8n·차단 확인
2. API 키 침해	키 폐기·재발급·IP 차단	n8n 직접 실행 성공
3. 킬체인	3단계 상관분석·pfSense 차단	CRITICAL/WARNING 분기 확인

주요 주소

구성	주소/포트
API / Web	5000 / 5001
Middleware	8000 / 9100
Graylog / GELF	9000 / 12201
n8n / MySQL	5678 / 3306
Wazuh / pfSense	1514·1515·443 / 172.16.50.2

2. 설치·실행 절차

새 PC 또는 VM에서 README 순서만 보고 재현할 수 있도록 설치 순서를 고정했습니다.

PowerShell 실행 순서

```
git clone https://github.com/rykim1217/network-team03.git
cd network-team03
Copy-Item .env.example .env
python -m pip install -r requirements.txt -r web-app/requirements.txt
.₩start-enginesquare.ps1
```

실행 전 확인

- Docker Desktop과 MySQL·Middleware·Graylog·Wazuh·n8n이 먼저 실행되어야 합니다.
- 실제 수집 키는 로컬 .env에만 입력하고 .env는 Git에 올리지 않습니다.
- 시나리오 3의 실제 네트워크 차단까지 보려면 pfSense VM과 방화벽 별칭이 필요합니다.
- 실행 후 API /health, 웹 /dashboard, Graylog /alerts, n8n 실행 기록을 순서대로 확인합니다.

설정 파일 관리

- Graylog 집계·알림 규칙, Wazuh 규칙, n8n workflow JSON은 저장소에 포함했습니다.
- Discord는 Incoming Webhook을 사용하며 저장소에는 placeholder만 남겼습니다.
- runtime log, security_events.jsonl, .env는 .gitignore로 제외했습니다.

3. 개인 기여 범위

팀 공통 구현을 개인 단독 구현으로 표현하지 않고, 커밋·수정·검증 기준으로 개인 범위를 분리했습니다.

구분	내용
팀 공통	Flask API, 공통 웹앱, MySQL 모델, Graylog/Wazuh/n8n 기본 구조, Middleware WAL
개인 수정	n8n 시나리오 1~3 Discord Incoming Webhook 전환, workflow 민감정보 placeholder화
개인 반영	시나리오 3 n8n·웹 UI 변경 반영, 루트 테스트를 현재 SecurityStore 구조에 맞게 보완
개인 검증	API/Web smoke, 브루트포스, 2만 건 spike, 시나리오 2 직접 실행, 시나리오 3 CRITICAL/WARNING

커밋 근거

rykim1217 작성 커밋에서 `문구수정`, `n8n파일 교체`, Discord 봇/임베드 알림 최신화, 시나리오 3 workflow·웹 UI 반영 내역을 확인할 수 있습니다. 팀 원본 저장소는 README와 함께 출처로 연결했습니다.

보안 검사

- tracked .env: 없음
- 실제 Discord webhook URL: 없음
- 구형 Discord Bot 토큰·Authorization: 없음
- 로컬 .env·runtime log·security_events.jsonl: ignore 확인

4. 실행·검증 결과

검증	결과	판정
API /health	200, status=ok	완료
Web smoke	200 응답	완료
루트 테스트	14 passed, 2 warnings	완료
Scenario 1	Graylog Alert → n8n 성공 → IP 차단	완료
Scenario 2 직접 실행	키 폐기·재발급·IP 차단·알림 노드 진행	완료
Scenario 3 CRITICAL	pfSense 별칭·인시던트 확인	완료
Scenario 3 WARNING	CRITICAL 격상 없음	완료

2만 건 스파이크 실측

지표	값
최종 received / forwarded	20,005 / 20,005
dropped / DLQ	0 / 0
최종 queue lag	0
peak queue lag	19,051
peak outbound	200 / limit 500

주의: 스크립트의 sent 20,004와 최종 received 20,005가 1건 다릅니다. 테스트 중 백그라운드 이벤트 1건이 합류한 것으로 판단하여 엄격한 exact-count 검사는 실패했지만, 최종 drain·유실 0·lag 0은 확인했습니다.

5. 문제 해결과 개선

Middleware metrics 지연

문제: 첫 2만 건 테스트 전에 /metrics가 멈춤 → 원인: ingestor/worker 정체 → 해결: 두 컨테이너 재시작 후 health·metrics 확인 → 결과: 최종 lag 0, dropped 0.

Discord 방식 불일치

문제: 팀 공용 봇 방식과 개인 채널이 혼용됨 → 원인: n8n 노드 설정 혼합 → 해결: Incoming Webhook으로 통일하고 저장소에는 placeholder만 유지 → 결과: 개인 채널 대상 알림 구조로 정리.

시나리오 2 production webhook

n8n 편집기 직접 실행은 성공했습니다. 다만 Graylog가 호출하는 운영용 POST /webhook/scenario2-api-key-compromised에서 404가 한 번 관찰되어, 제출 전 workflow Active 상태와 Graylog URL을 다시 맞춰야 합니다.

느낀 점

보안 자동화는 탐지 규칙만 만드는 일이 아니라 수집·분석·대응·복구·비밀정보 관리까지 실제 환경에서 끝까지 확인해야 완성됩니다. 다음 개선으로는 run id를 도입해 백그라운드 이벤트가 대량 테스트 수량에 섞이지 않도록 하겠습니다.

6. 제출 전 체크리스트

- 이름과 정확한 프로젝트 기간 입력
- 개인 GitHub 저장소에 README·portfolio·workflow·규칙 파일 push
- API health, 대시보드, Graylog, n8n 성공 화면 캡처
- 시나리오 1 또는 3 재현 화면 캡처
- pfSense 차단 별칭과 복구 화면 캡처
- 2만 건 최종 metrics에서 dropped=0, lag=0 캡처
- GitHub에서 .env·실제 키·토큰·웹훅이 보이지 않는지 최종 검색
- 시나리오 2 production webhook 활성화 후 최종 smoke test
- 이력서·자기소개서 초안의 [입력 필요] 부분 수정

제출 문구

README대로 설치 → 동작 확인 → 담당 작업·개선점 정리 완료

첨부 문서

portfolio/개인_포트폴리오_정리.md · portfolio/검증_기록.md · portfolio/이력서_프로젝트_항목.md · portfolio/자기소개서_기본본.md

EngineSquare
증적 캡처 부록

개인 포트폴리오 제출용 · 시나리오별 검증 증적

공통 자료와 시나리오 1·2·3 증적을 서로 섞지 않고 각각 묶었습니다.

공통 자료

개인 저장소 연결, 프로젝트 구조, 실행 파일, 루트 테스트처럼 특정 시나리오에 속하지 않는 자료입니다.

공통 01. 개인 저장소·팀 원본 연결

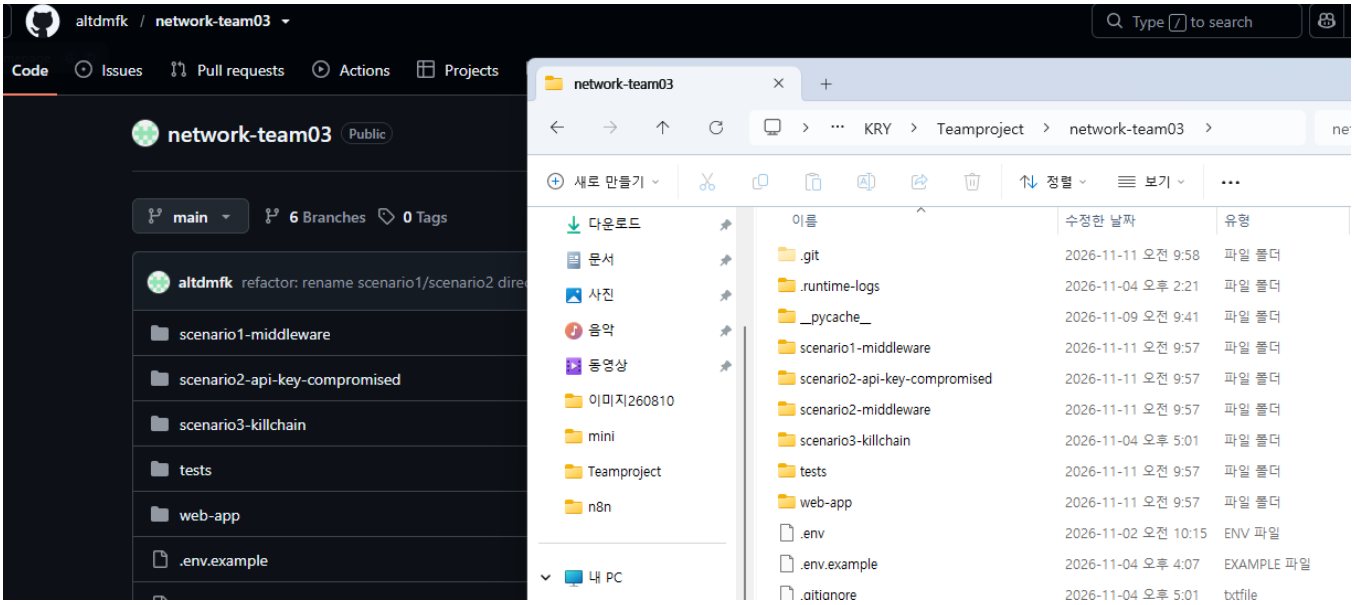
```
D:\KRY\Teamproject\network-team03> git remote -v
origin https://github.com/rykim1217/network-team03.git (fetch)
origin https://github.com/rykim1217/network-team03.git (push)
upstream      https://github.com/altdmfk/network-team03.git (fetch)
upstream      https://github.com/altdmfk/network-team03.git (push)

D:\KRY\Teamproject\network-team03> git status --short
 M README.md
 M scenario1-middleware/SCENARIO1_SETUP_GUIDE.md
 M scenario1-middleware/n8n/scenario1_bruteforce_response_workflow.json
 M "scenario2-api-key-compromised/n8n/API Key Compromised Response.json"
 M scenario3-killchain/n8n-workflows/scenario3-killchain-response.json
 M tests/test_app.py
?? portfolio/

D:\KRY\Teamproject\network-team03>
```

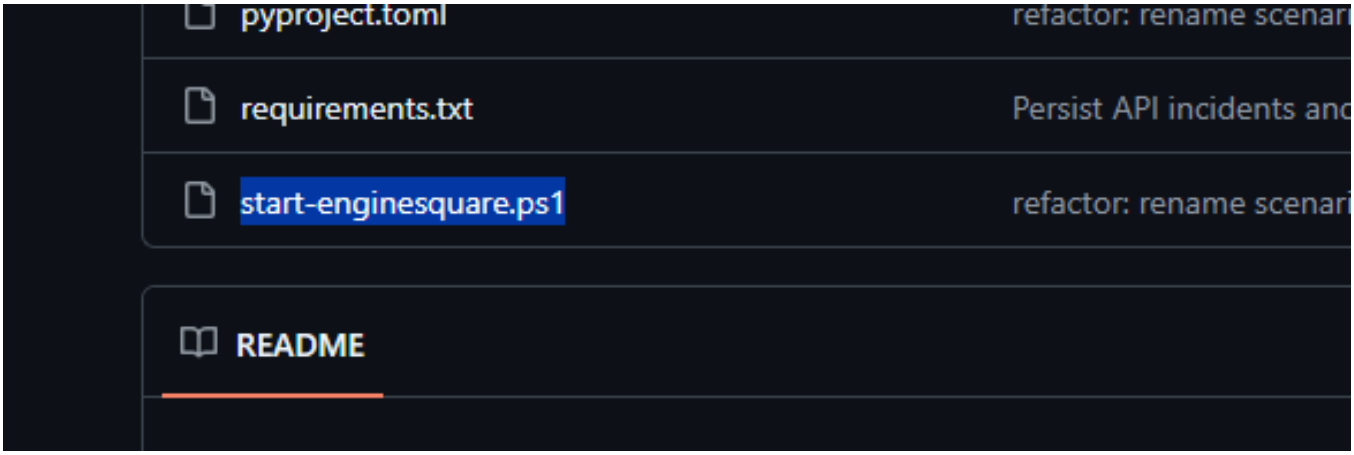
origin은 개인 저장소, upstream은 팀 원본 저장소로 연결되어 있음을 확인한 실제 터미널 결과입니다.

공통 02. 전체 프로젝트 구조



팀 저장소와 로컬 프로젝트에 시나리오 1·2·3 전체 구조가 내려와 있는 화면입니다.

공통 03. 실행·설정 파일



pyproject.toml, requirements.txt, start-enginesquare.ps1, README가 저장소에 포함된 화면입니다.

공통 04. 루트 자동화 테스트

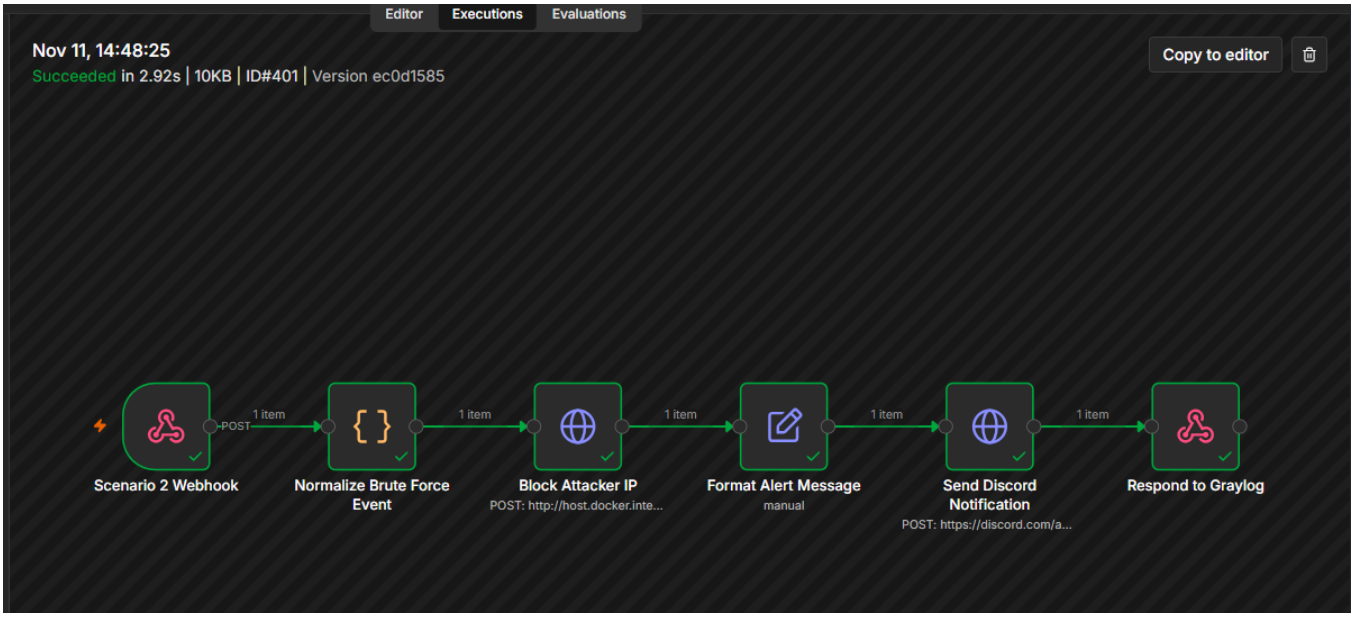


실제 로컬에서 python -m pytest -q tests -p no:warnings를 실행한 결과이며 14개 테스트가 통과했습니다.

시나리오 1 · 브루트포스 대응

시나리오 1은 공격 → Graylog 집계 → n8n 자동 차단 → Discord 알림 → 후속 403 순서로 묶었습니다. 2만 건 스파이크 검증은 마지막에 배치했습니다.

시나리오 1-1. n8n 자동 대응 실행



n8n 실행 ID 401 성공 화면으로 Normalize Brute Force Event, 공격자 IP 차단, Discord 알림, Graylog 응답 흐름이 모두 성공했습니다.

주석: 원래 시나리오 1 허니토큰 워크플로가 빠지고 그 자리에 시나리오 2 워크플로가 들어간 상태입니다.
실행 ID 401의 실제 흐름은 시나리오 1 Brute Force 대응(차단·Discord·Graylog 응답)입니다.

시나리오 1-2. 브루트포스 → 자동 차단

```
D:\KRY\Teamproject\network-team03> python scenario1-middleware\simulators\demo_bruteforce_attack.py --ip 1

[SCENARIO 1] Brute force attack & SOAR auto-response
attacker IP: 198.51.100.201 | failed attempts: 5

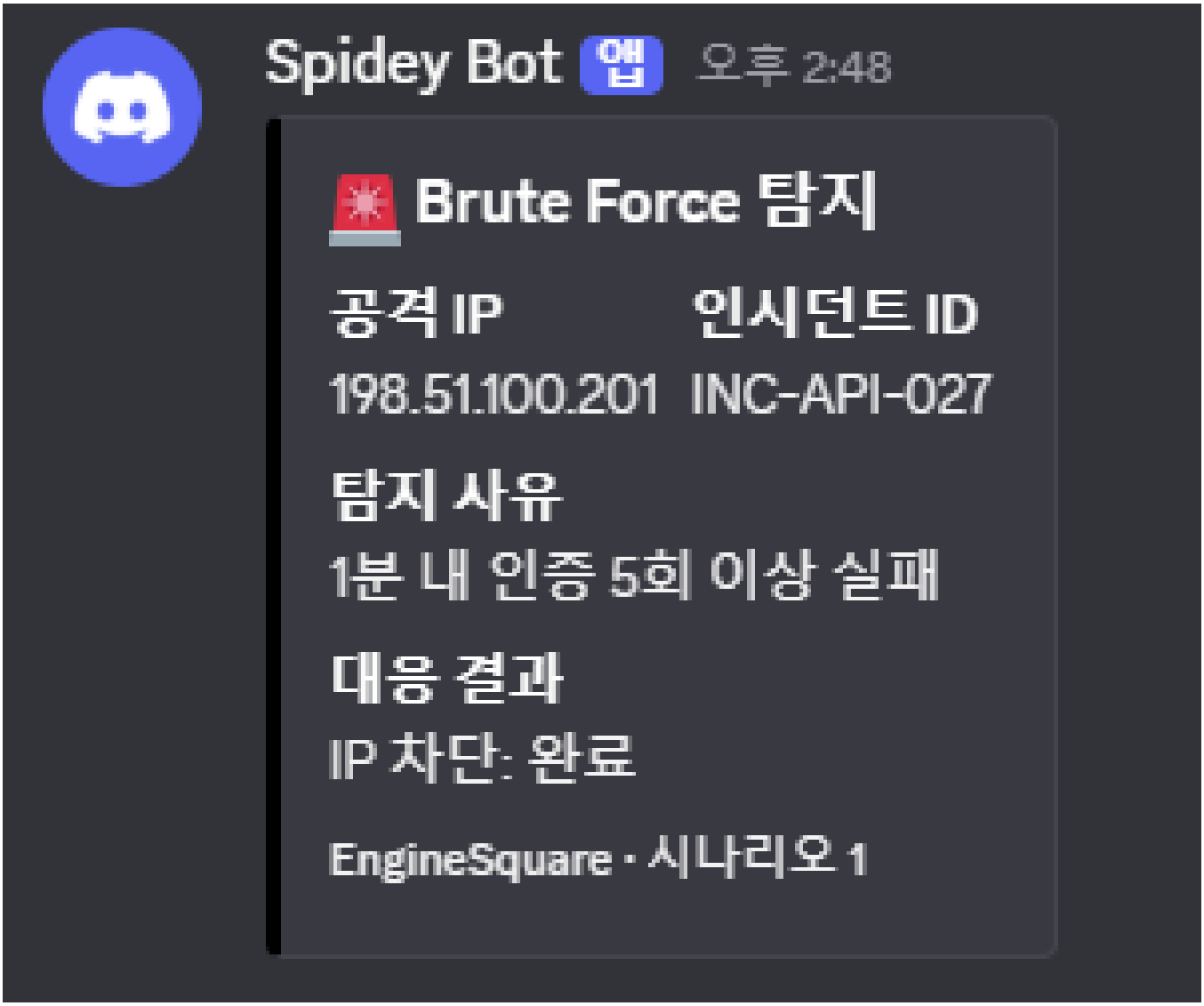
[1/5] API response: HTTP 401
[2/5] API response: HTTP 401
[3/5] API response: HTTP 401
[4/5] API response: HTTP 401
[5/5] API response: HTTP 401

[SOAR] 5 security logs buffered and forwarded to Graylog
[SUCCESS] n8n SOAR registered attacker IP in the block list
[FINAL VERIFICATION] API response: HTTP 403
[BLOCK CONFIRMED] Client IP has been blocked

SUMMARY: 5 failed attempts -> Graylog alert -> n8n auto-block -> HTTP 403
D:\KRY\Teamproject\network-team03>
```

실제 실행: 잘못된 키 5회(HTTP 401) → Graylog 집계 → n8n 차단 → 후속 요청 HTTP 403.

시나리오 1-3. Discord 탐지 알림



실제 Discord 채널에 도착한 Brute Force 탐지 알림으로 공격 IP, 인시던트 ID, 탐지 사유, IP 차단 완료가 표시됩니다.

시나리오 1-4. 2만 건 스파이크 검증

```
D:\KRY\Teamproject\network-team03> python simulators/spike_test_demo.py --spike-count 20000

[SCENARIO 1] Middleware spike test final metrics
sent requests                  = 20,004
final received / forwarded     = 20,005 / 20,005
dropped                        = 0
permanently dropped / DLQ      = 0 / 0
invariant violations           = 0
peak queue lag                 = 19,051
final queue lag                = 0
outbound / configured limit    = 200 / 500

RESULT: zero loss and queue fully drained
NOTE: one background event was included in the same aggregate run
D:\KRY\Teamproject\network-team03>
```

실측 지표: received/forwarded 20,005건, dropped 0, DLQ 0, 최종 queue lag 0. 백그라운드 이벤트 1건 포함 사실도 함께 기록했습니다.

시나리오 2 · API Key Compromised

시나리오 2는 정상 API 요청 → 키 폐기 후 거부 → Discord 대응 알림 → n8n 자동 대응 → 관리자 보안 기능 순서로 묶었습니다. 키 값 자체는 캡처에서 제거했습니다.

시나리오 2-1. n8n 자동 대응 실행



n8n 실행 ID 400 성공 화면으로 API 키 폐기·재발급, IP 차단, 인시던트 생성, 대시보드·Discord 알림 흐름이 성공했습니다.

시나리오 2-2. 폐기 전 API 요청 성공

200 OK • 77 ms • 234 B


```
{
    "data": "demo response",
    "message": "authorized",
    "request_count": 1
}
```


Postman 실제 응답: HTTP 200 OK, message: authorized. API 키 값과 요청 헤더는 개인정보 보호를 위해 제외했습니다.

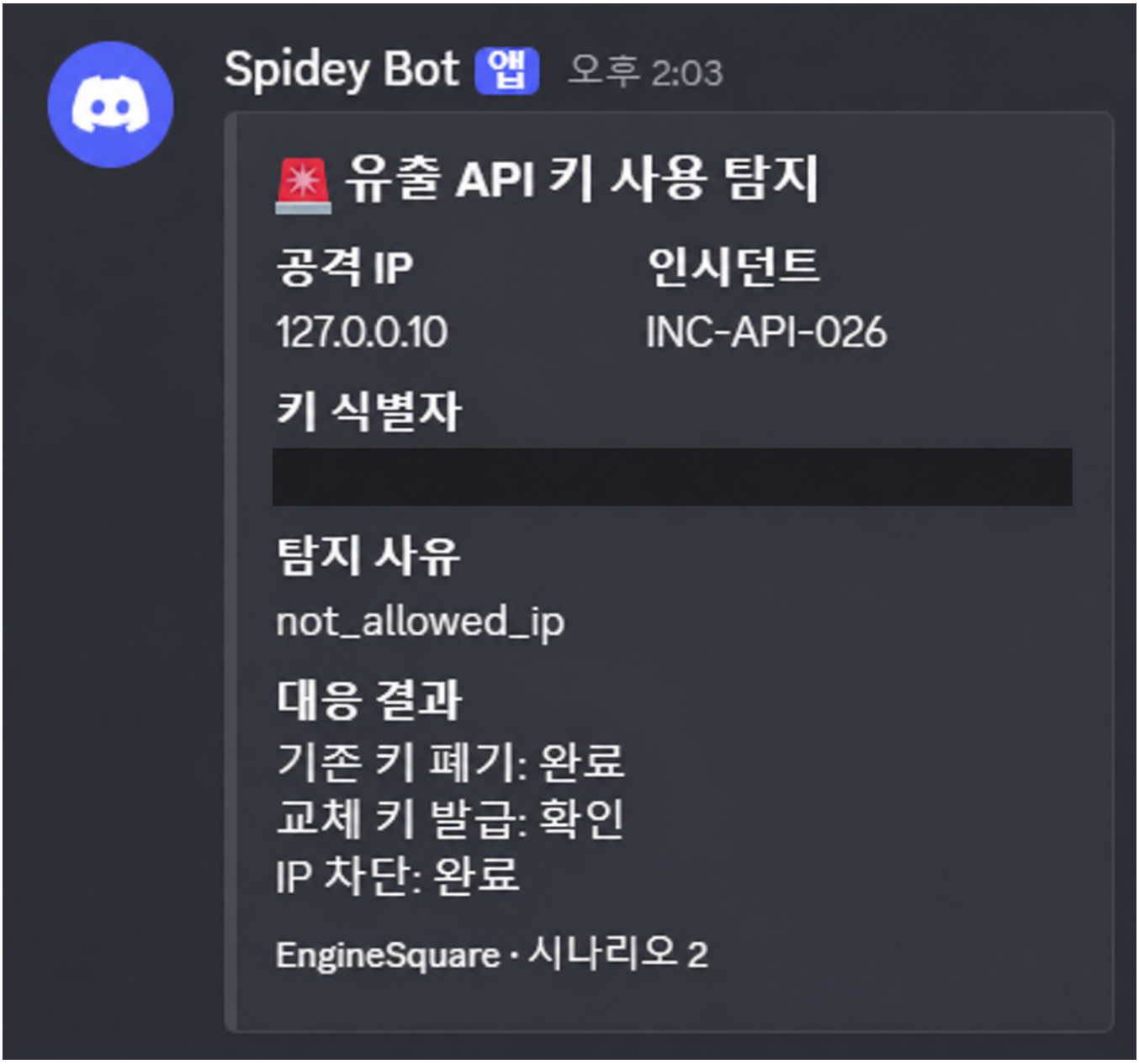
시나리오 2-3. 키 폐기 후 API 요청 거부

401 Unauthorized • 274 ms • 238 B

```
{
    "error": "request_rejected",
    "message": "API key is revoked"
}
```

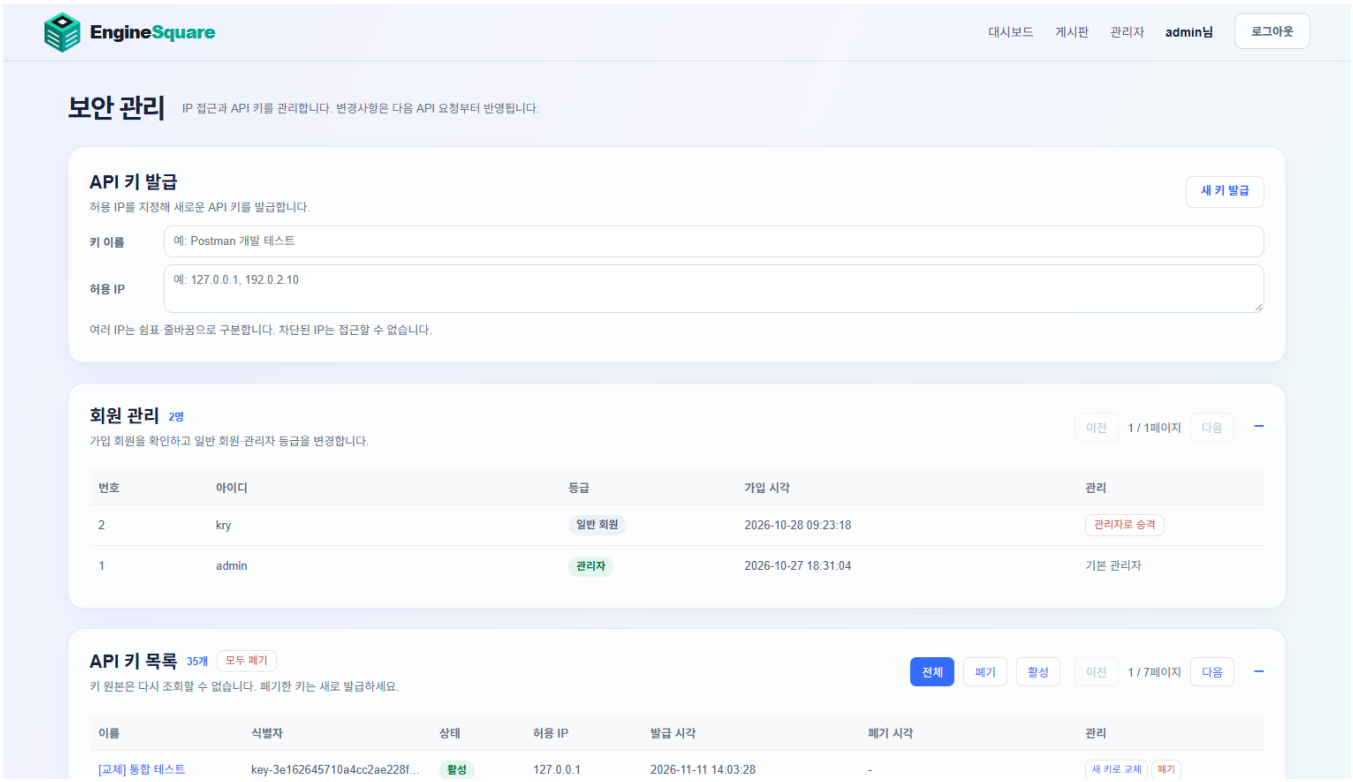
Postman 실제 응답: HTTP 401 Unauthorized, request_rejected, API key is revoked.

시나리오 2-4. Discord 자동 대응 알림



실제 Discord 알림으로 기존 키 폐기, 교체 키 발급 확인, 공격 IP 차단 완료가 표시됩니다. 키 식별자는 가렸습니다.

시나리오 2-5. 관리자 보안 기능

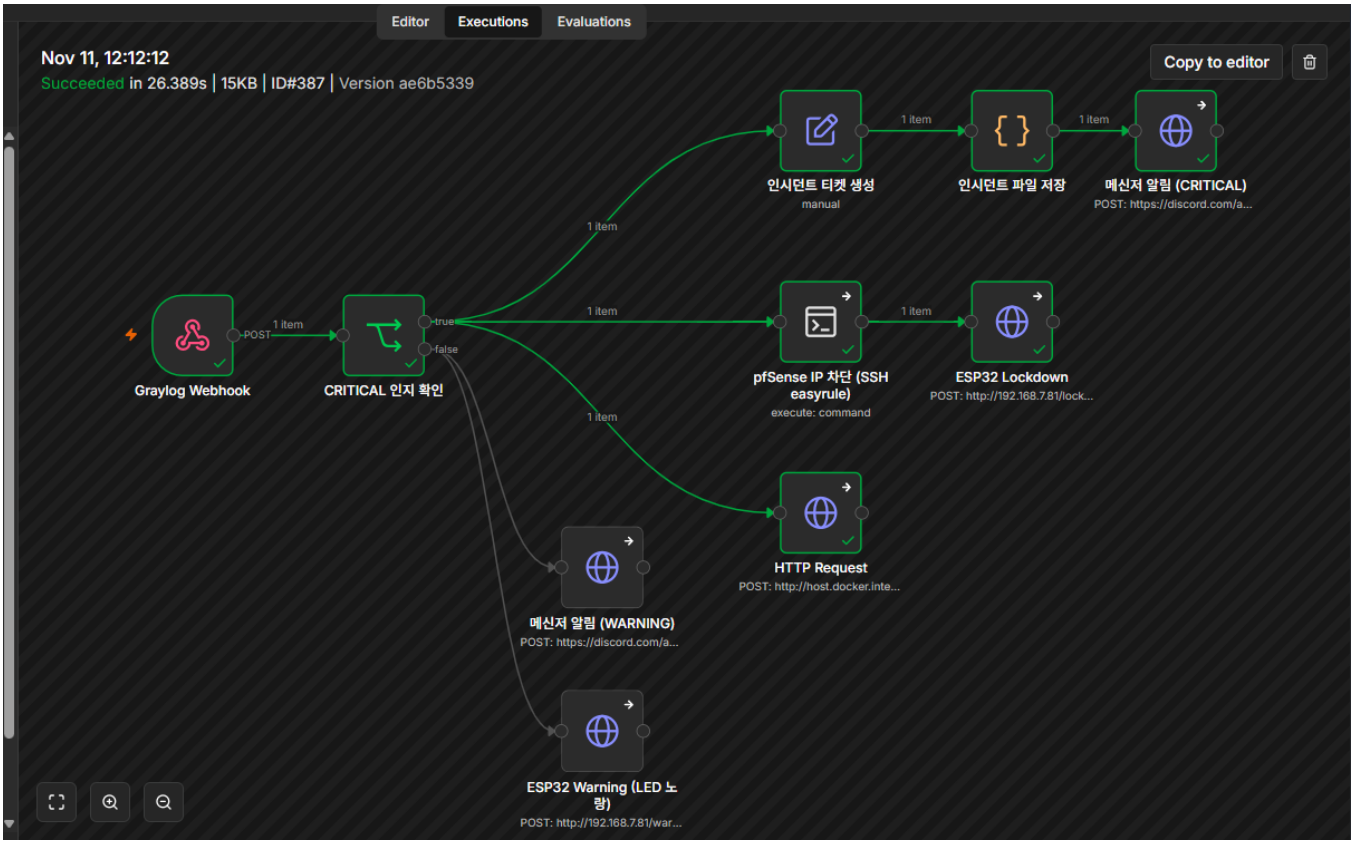


사용자가 제공한 EngineSquare 보안 관리 화면으로 API 키 발급, 허용 IP, 회원 관리, API 키 목록과 폐기 기능을 확인할 수 있습니다.

시나리오 3 · Kill Chain 대응

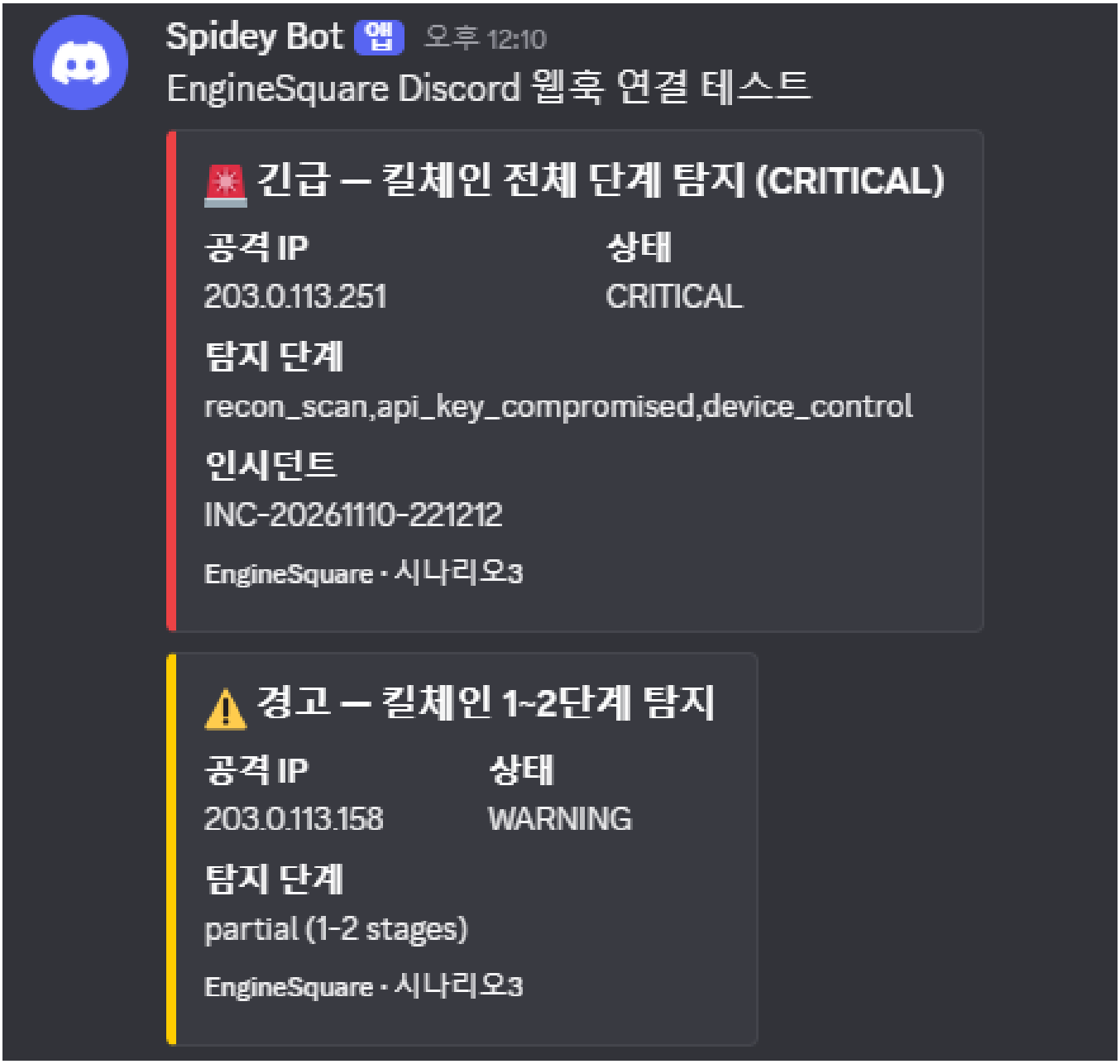
시나리오 3은 Discord CRITICAL/WARNING 결과 → n8n 분기 실행 → Graylog 탐지 규칙 → Wazuh 계층 → pfSense 차단 결과 순서로 묶었습니다.

시나리오 3-1. n8n Kill Chain 실행



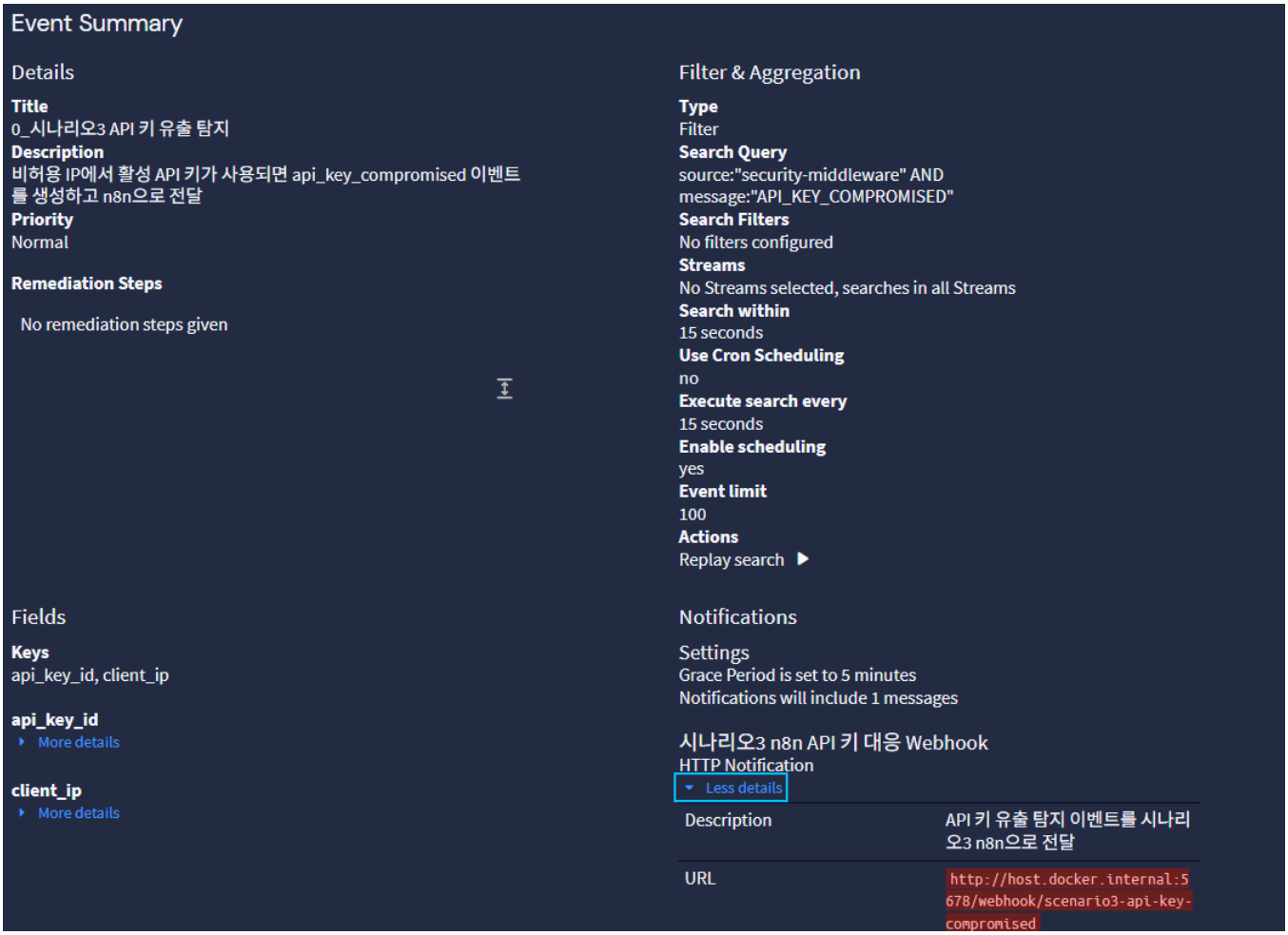
n8n 실행 ID 387 성공 화면으로 CRITICAL 확인, 인시던트 생성, pfSense IP 차단, ESP32 lockdown, Discord 알림 분기를 확인했습니다.

시나리오 3-2. Discord CRITICAL·WARNING 결과



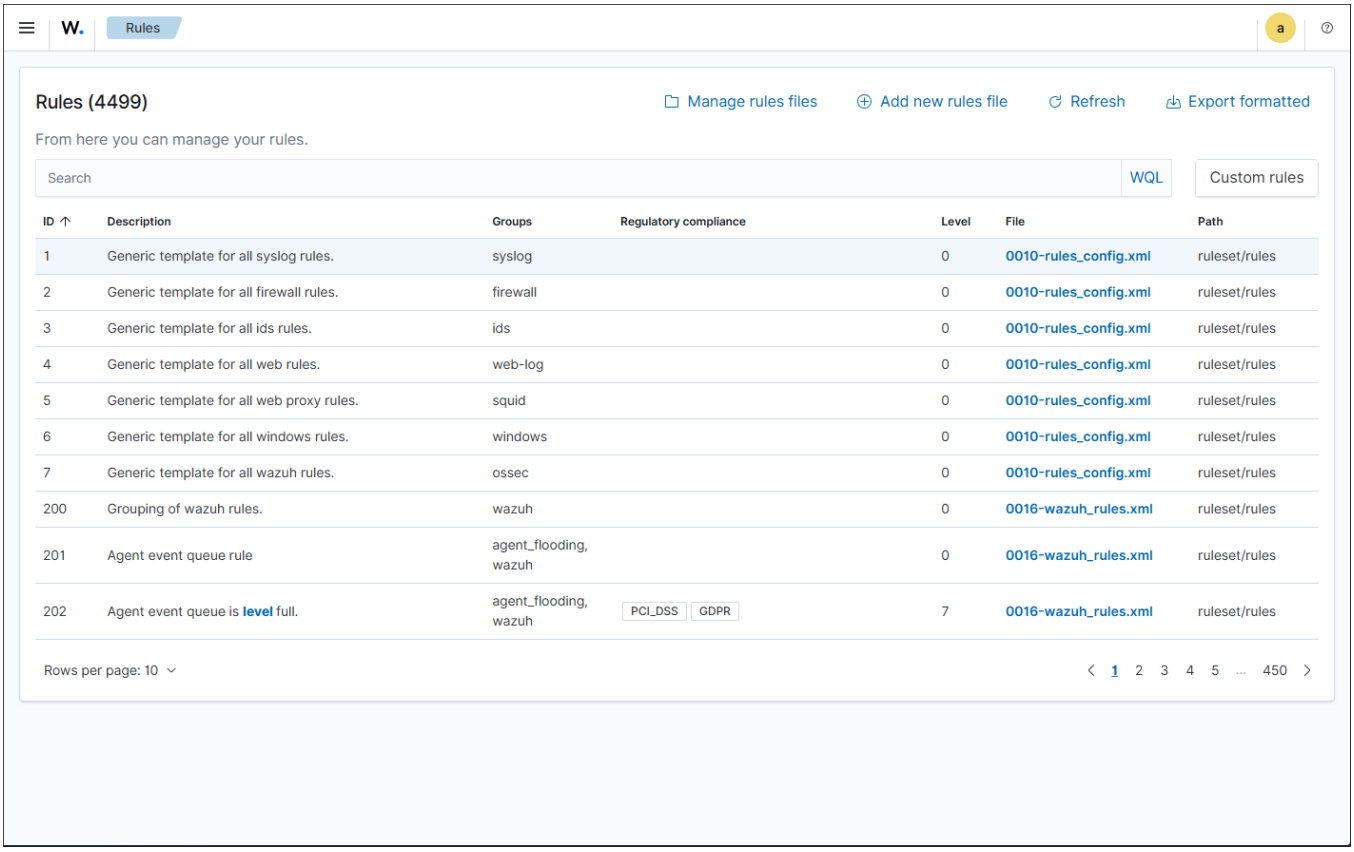
실제 Discord 알림으로 3단계 CRITICAL과 1~2단계 WARNING 결과를 함께 확인한 화면입니다.

시나리오 3-3. Graylog 탐지 규칙·n8n 전달



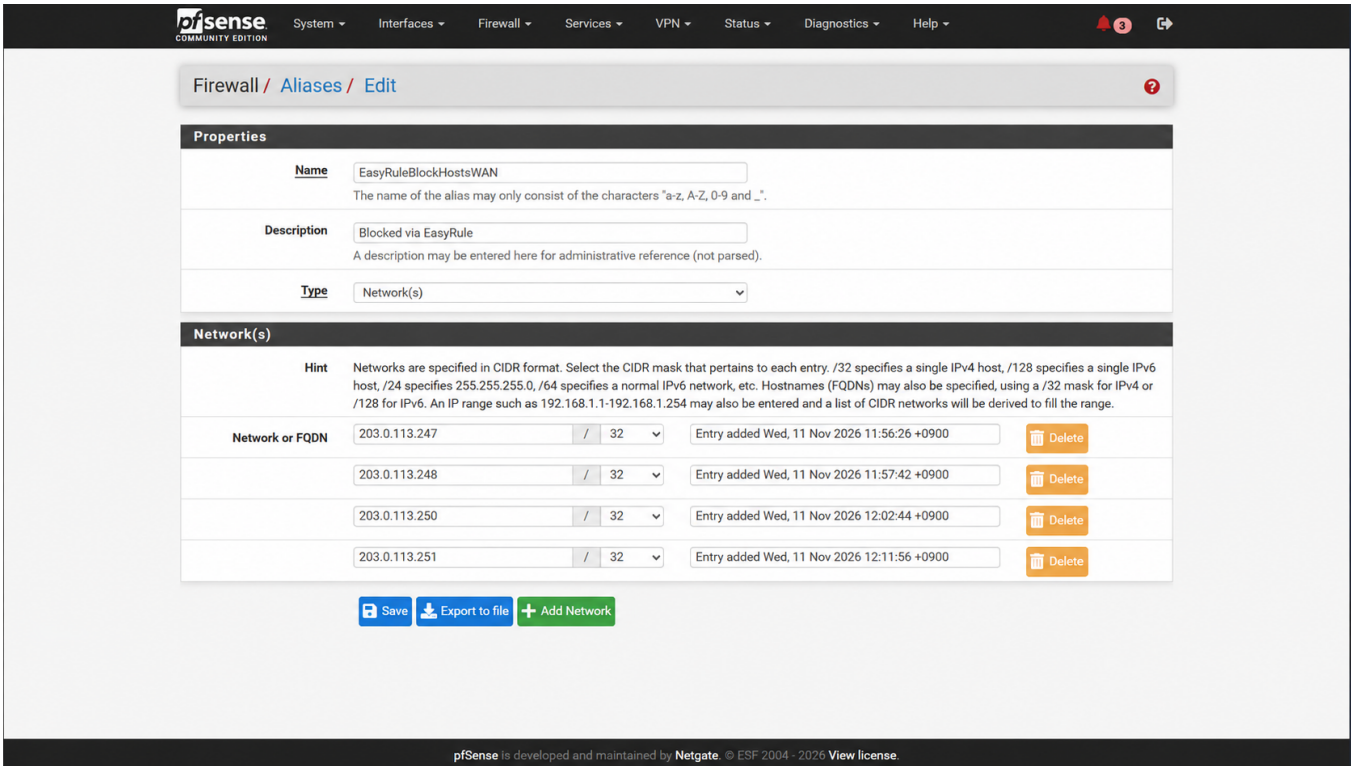
API 키 침해 이벤트를 탐지하고 내부 n8n 웹훅으로 전달하는 Graylog Event Definition 화면입니다.

시나리오 3-4. Wazuh 규칙 관리



Wazuh 규칙 관리 화면으로 보안 이벤트 수집 계층을 확인한 화면입니다.

시나리오 3-5. pfSense 차단 결과



EasyRuleBlockHostsWAN 별칭에 시나리오 테스트 IP가 /32로 반영된 실제 화면입니다. 마우스 커서는 제거했습니다.